



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

AIML for Robotics

Module 5

Path Planning

Section Faculty:

Raja vadhana Prabhakar

The course owner and the authors of this course deck, Prof. Raja vadhana and Dr.Sangeetha, are gratefully acknowledging all authors who made their course materials freely available online.

☐ Introduction

- ☐ Types of Robots
- ☐ Mobile Robots – Use case of Programmable solutions

☐ Components of Programmable Robotic System

- ☐ Perception
- ☐ Representation
- ☐ Motion

☐ Localization

- ☐ Taxonomy
- ☐ Types

☐ Mapping

- ☐ Feature Based
- ☐ Grid Based

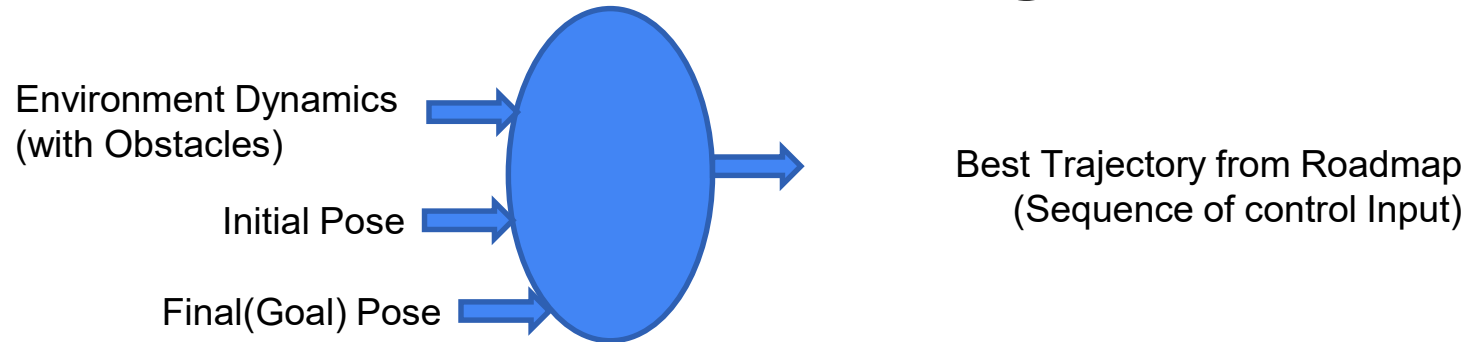
☐ Path Planning

- ☐ Graph Based
- ☐ Sample Based
- ☐ Evolutional Techniques

☐ Human – Robot Interaction

- ☐ Ethics
- ☐ Introduction to Visual Bot
- ☐ Introduction to Chat Bot

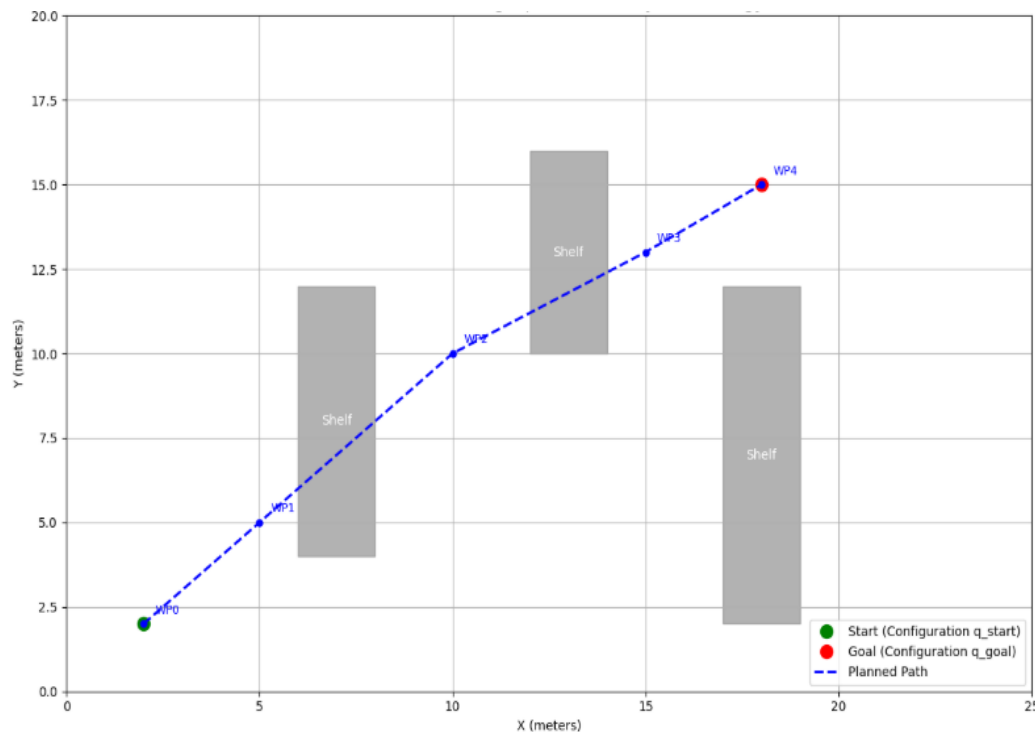
Robot Path Planning



Planning Agents



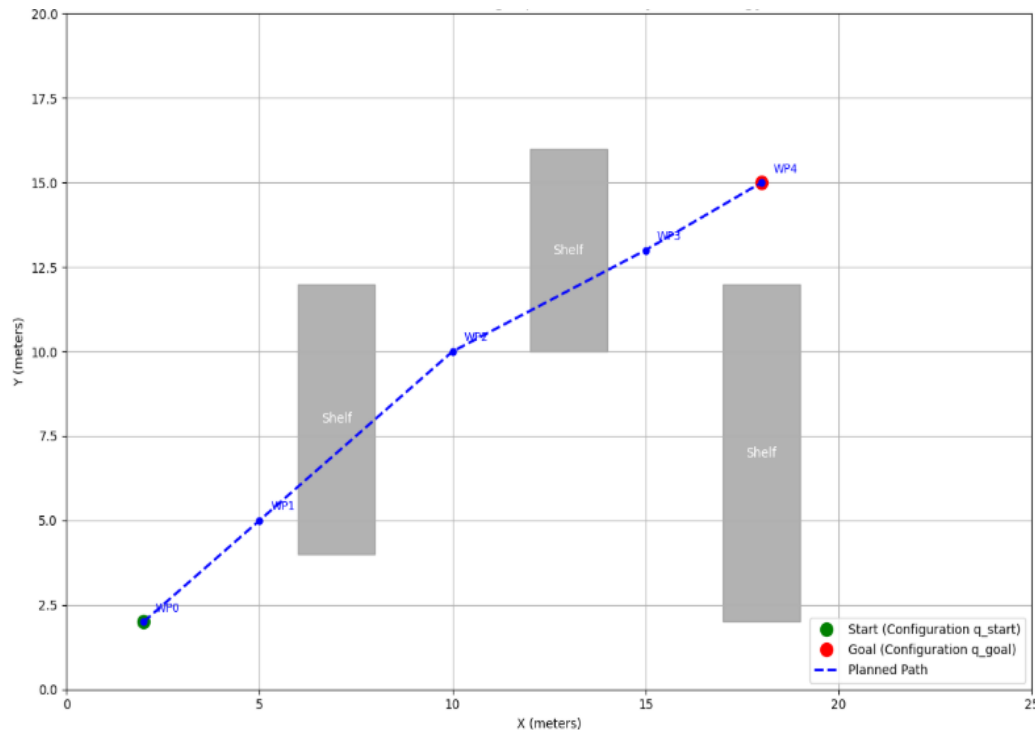
- ✓ Configuration Space
- ✓ Collision
- ✓ Map/Workspace
- ✓ Waypoints:
 $(2,2) \rightarrow (5,5) \rightarrow (10,10) \rightarrow (15,13) \rightarrow (18,15)$
- ✓ Path
- ✓ Trajectory
- ✓ Roadmap
- ✓ Sampling



Planning Agents



- Path Length
- Cost- Heuristics
- Obstacle Proximity
- Smoothness
- Clearance
- Turn Count
- Train vs Test Accuracy



Planning Agents



Eg., At Path Level: $(1,1) \rightarrow (2,2) \rightarrow (3,3)$

- Clearance

$$= \min\{d(p1,o), d(p2,o), d(p3,o)\}$$

$$= \min\{5, 3, 1\}$$

$$= 1 \text{ unit}$$

- Obstacle Proximity

$$3s \quad (2,2) \quad (2.5, 3.3) \quad 1.8m < 0.5m$$

$$5s \quad (2.5,3) \quad (2.5, 3.3) \quad 0.3m < 0.5m$$

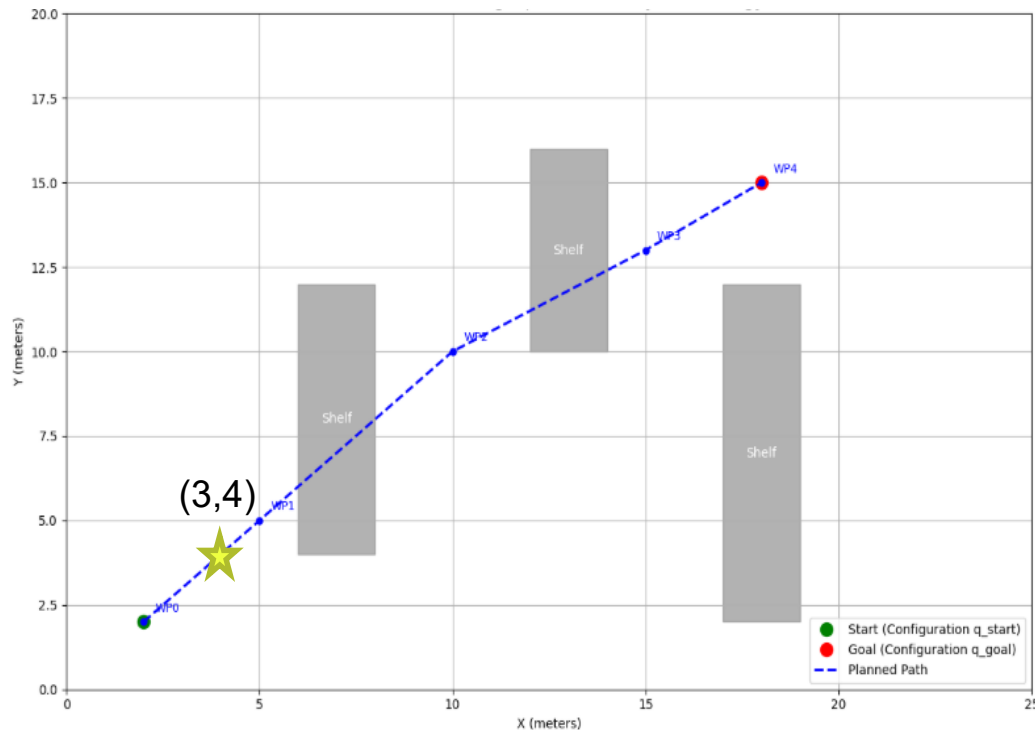
- Smoothness

$$P1: (1,1,30) \rightarrow (2,2,30) \rightarrow (3,3, 60)$$

$$= 0 + 30 = 30$$

$$P2: (1,1,30) \rightarrow (2,2,30) \rightarrow (3,3, 20)$$

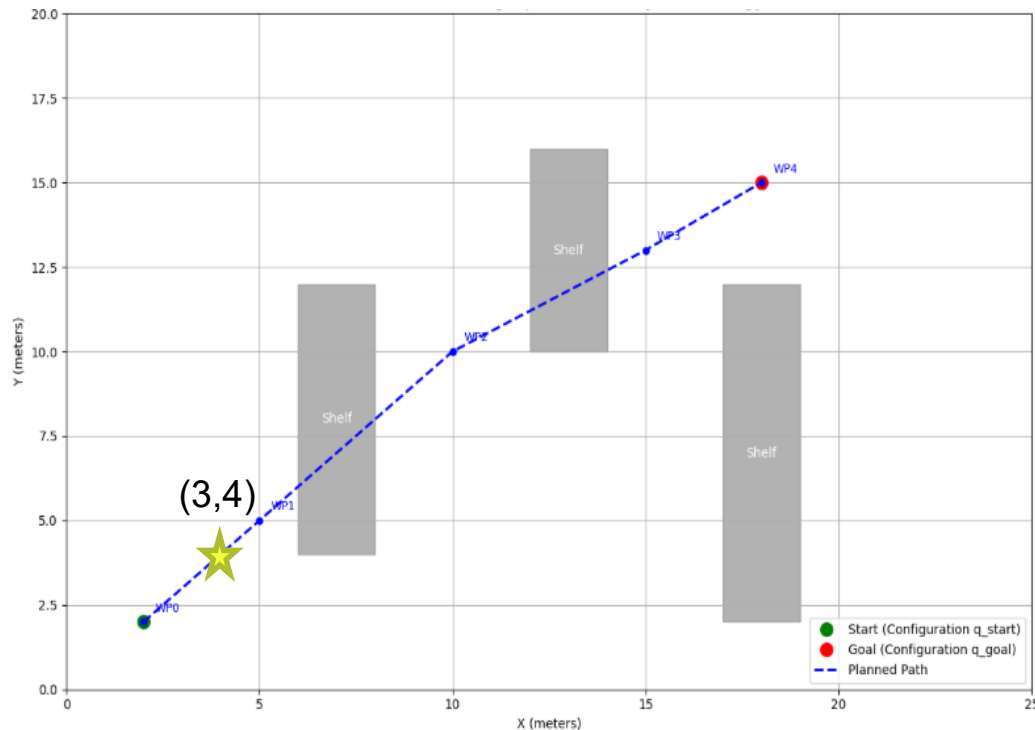
$$= 0 + 10 = 10$$



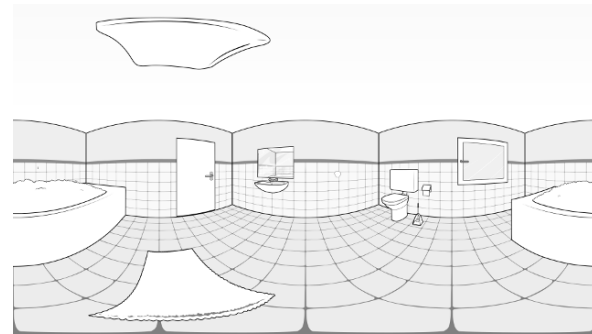
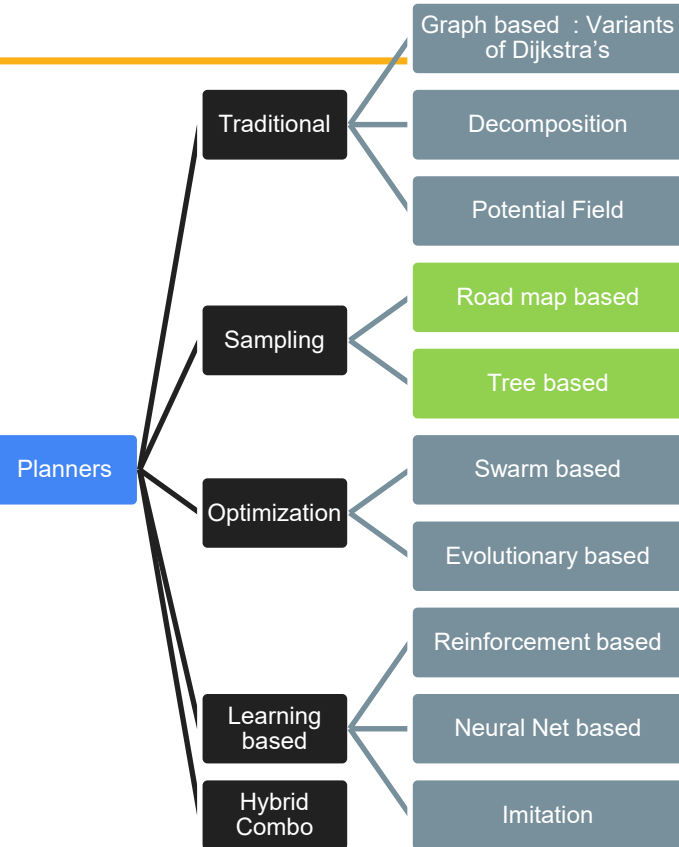
Planning Procedure



1. Initialize
2. Explore the Environment
3. Check the Constraints
4. Update the Learning
5. Converge/ Repeat



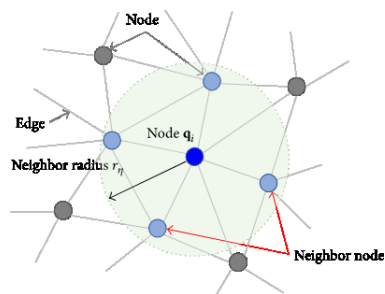
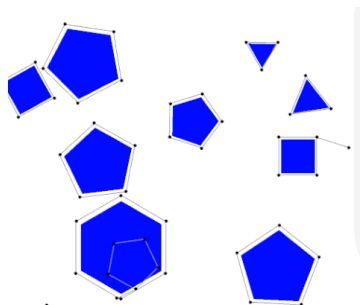
Taxonomy of Planning Problems



- ❖ Application
- ❖ Environment
- ❖ Constraints

Source Credit : <https://www.deviantart.com/>

Sampling Based Path Planning



Observation : RGG in incremental version



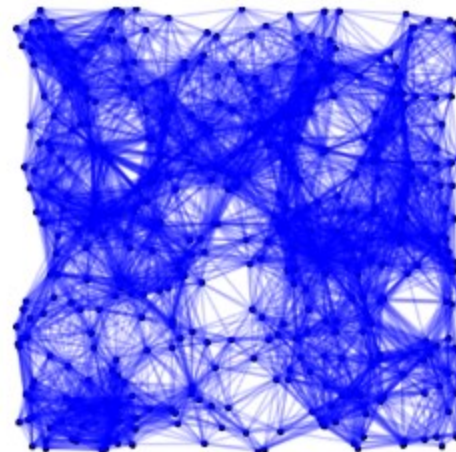
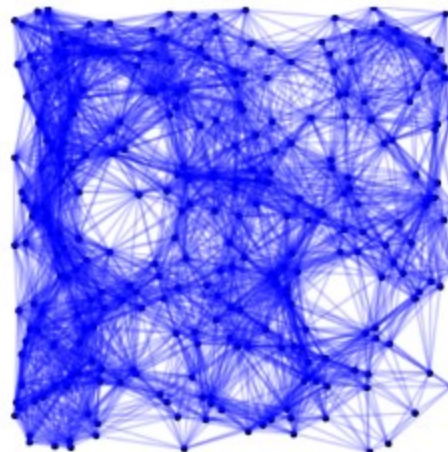
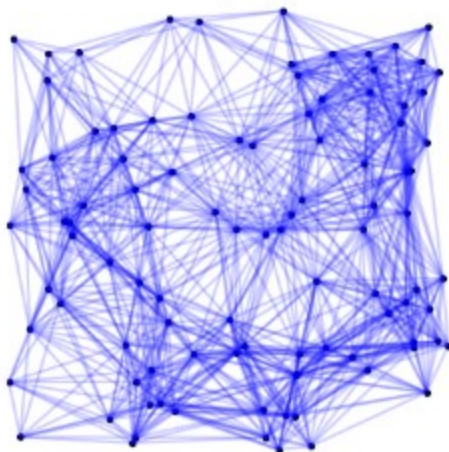
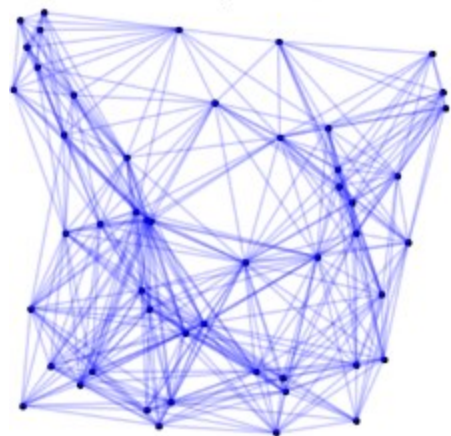
RGG Evolution with Increasing n and Scaled Radius

$n = 50, r \approx 0.42$

$n = 100, r \approx 0.32$

$n = 300, r \approx 0.21$

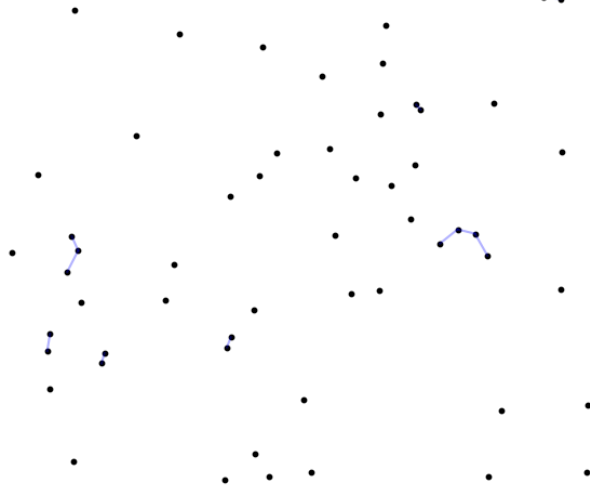
$n = 500, r \approx 0.17$



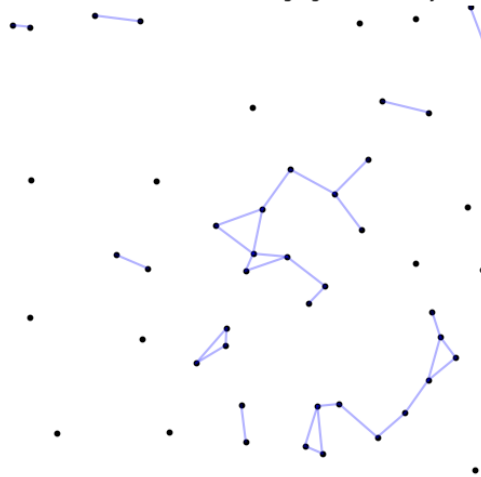
Performance : Asymptotically Optimal



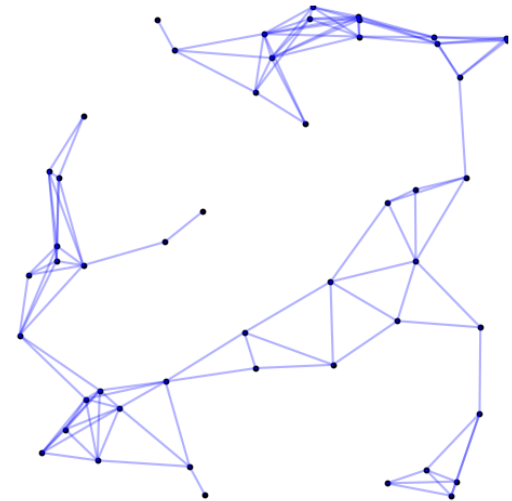
Below Percolation Threshold (Disconnected)



Around Threshold (Emerging Connectivity)



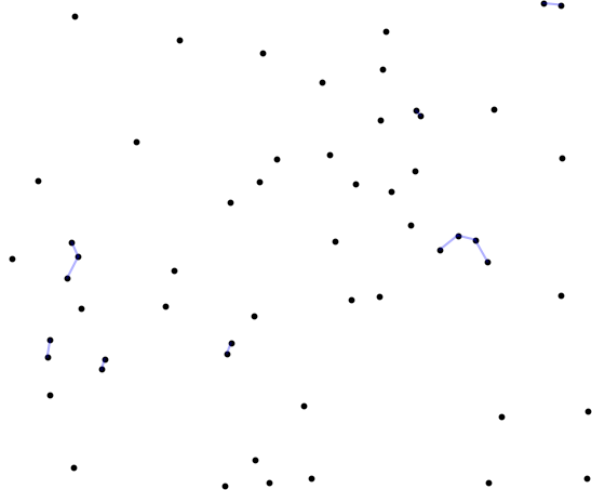
Above Threshold (Connected)



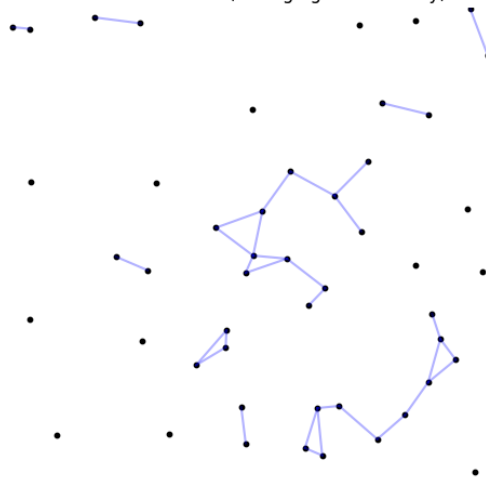
Performance : Probabilistically Complete



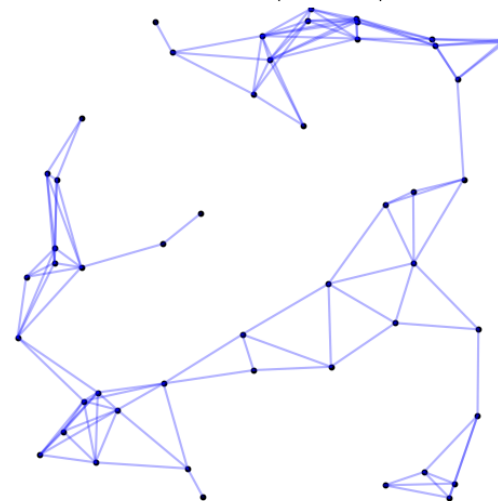
Below Percolation Threshold (Disconnected)



Around Threshold (Emerging Connectivity)



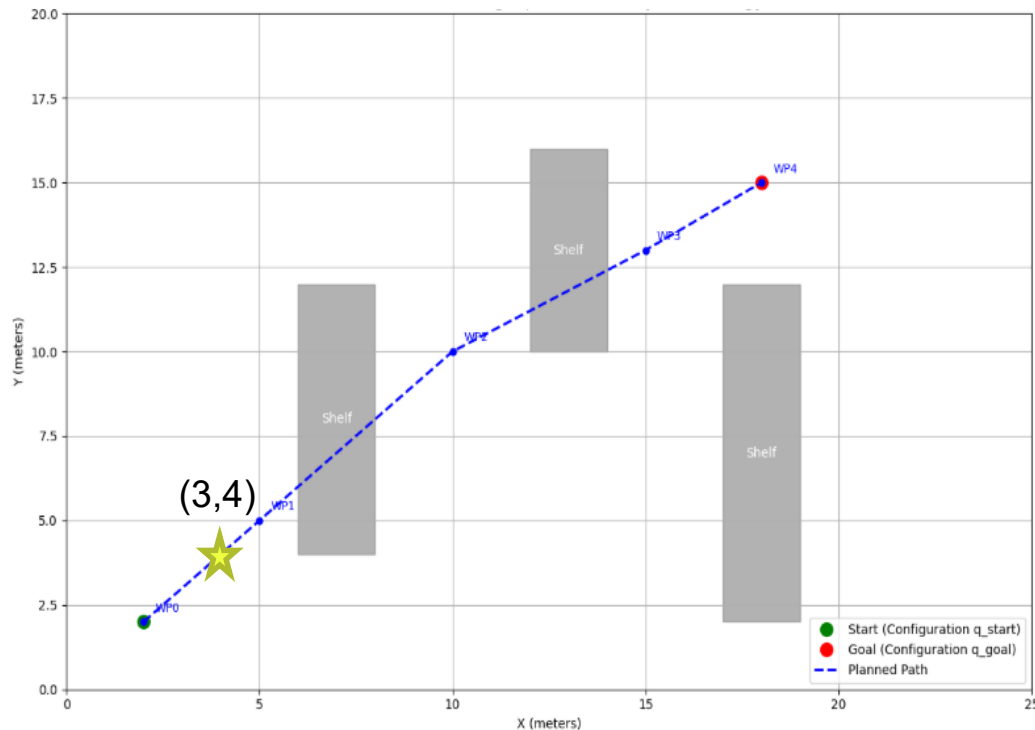
Above Threshold (Connected)



Planning Procedure



1. Initialize
2. Explore the Environment
3. Check the Constraints
4. Update the Learning
5. Converge/ Repeat



Planning Procedure



RRT

PRM

- | | | |
|----------------------------|------------------------------|------------------------------|
| 1. Initialize | 1. Initialize | 1. Initialize |
| 2. Explore the Environment | 2. Generate Sample Particles | 2. Generate Sample Particles |
| 3. Check the Constraints | 3. Search Nearest Neighbor | 3. Search Nearest Neighbor |
| 4. Update the Learning | 4. Steer Collision Free Path | 4. Check Collision Free Path |
| 5. Converge/ Repeat | 5. Construct the Path | 5. Construct the Graph |
| | 6. Post Processing | 6. Query via Graph Search |
| | 7. Goal Test | 7. Goal Test |
| | 8. Repeat | 8. Repeat |

Rapidly Exploring Random Trees(RRT) – RRT



```
def RRT(START, GOAL):
```

```
    TREE = []
```

```
    TREE.add(START)
```

```
    DELTA = maximum distance between sampled node and nearest node.
```

```
    REPEAT n times:
```

```
        X = generateNewConfiguration()
```

```
        if X in FREE_SPACE:
```

```
            for nodes in TREE:
```

```
                Y = argmin(nodes, criteria=distance)
```

```
            if DIST(X, Y) < DELTA:
```

```
                Find a configuration Z that is at DELTA distance along the path from X to Y
```

```
                if TRAVERSABLE(X, Z):
```

```
                    Z.parent = Y
```

```
                    TREE.add(Z)
```

```
            else:
```

```
                if TRAVERSABLE(X, Y):
```

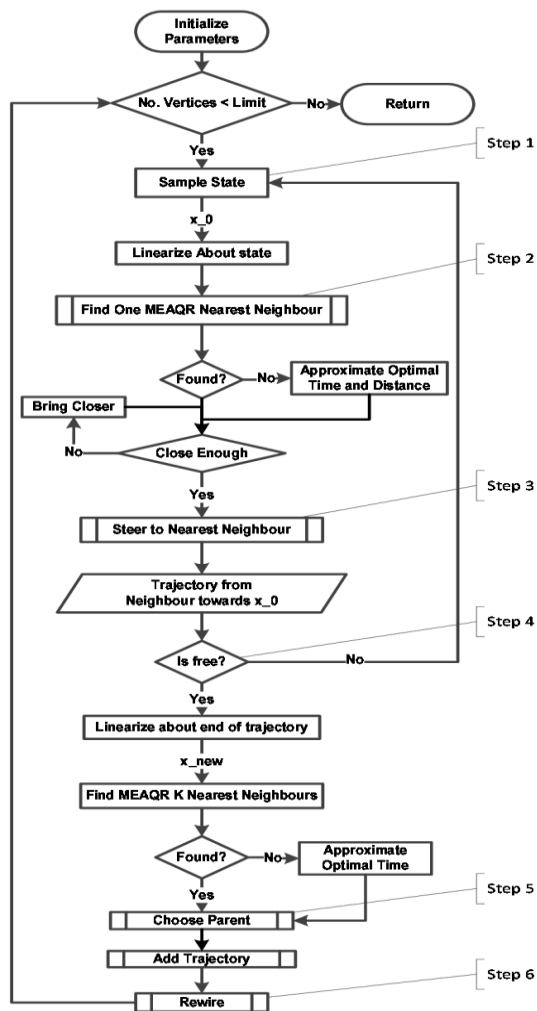
```
                    X.parent = Y
```

```
                    TREE.add(X)
```

```
        if X is GOAL:
```

```
            report "SUCCESS"
```

```
            break
```

def RRT_Star(V, E, gamma, c): # G = (V, E)

V = [x_init]

E = []

for i in range(1, n):

x_rand = SampleFree()

x_nearest = Nearest(V, E, x_rand)

x_new = Steer(x_nearest, x_rand)

if ObstacleFree(x_nearest, x_new):

X_near = Near(V, E, x_new, r) # $r = r(n) = \gamma * (\log(n) / n)^{1/d}$, d= dimension

list of all nodes at radius r

V.append(x_new)

x_min = x_nearest

c_min = cost(x_nearest) + c(Line(x_nearest, x_new))

the cost of the nearest node to new node to check whether to change paths

for x_near in X_near:

if CollisionFree(x_near, x_new) && Cost(x_near)+c(Line(x_near, x_new)) < c_min:

x_min = x_near

c_min = Cost(x_near) + c(Line(x_near, x_new))

update c_min, because we now need to find nodes that are closer than this new x_new

E.append([x_min, x_new])

for x_near in X_near:

if CollisionFree(x_new, x_near) && Cost(x_new) + c(Line(x_new, x_near)) < Cost(x_near):

x_parent = Parent(x_near)

E.remove([x_parent, x_near])

E.append([x_new, x_near])

remove all edges between nodes (in radius r) and their parents and join them with x_new this is the step of reconnection of edges

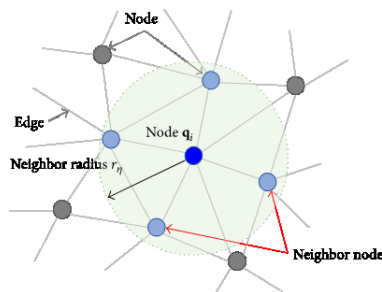
return V, E



RRT* Algorithm

RRT* Algorithm

1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Steer Collision Free Path
5. Construct the Path
6. Post Processing
7. Goal Test
8. Repeat



Algorithm 6: RRT*

```

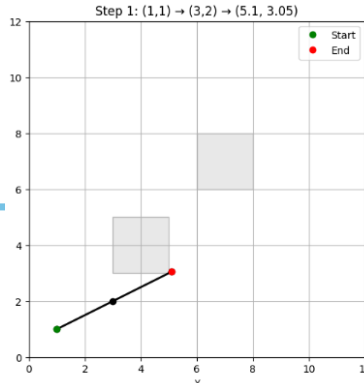
1  $V \leftarrow \{x_{init}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{rand} \leftarrow \text{SampleFree}_i;$ 
4    $x_{nearest} \leftarrow \text{Nearest}(G = (V, E), x_{rand});$ 
5    $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand});$ 
6   if  $\text{ObstacleFree}(x_{nearest}, x_{new})$  then
7      $X_{near} \leftarrow \text{Near}(G = (V, E), x_{new}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}(V))/\text{card}(V))^{1/d}, \eta\});$ 
8      $V \leftarrow V \cup \{x_{new}\};$ 
9      $x_{min} \leftarrow x_{nearest}; c_{min} \leftarrow \text{Cost}(x_{nearest}) + c(\text{Line}(x_{nearest}, x_{new}));$ 
10    foreach  $x_{near} \in X_{near}$  do // Connect along a minimum-cost path
11      if  $\text{CollisionFree}(x_{near}, x_{new}) \wedge \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new})) < c_{min}$  then
12         $x_{min} \leftarrow x_{near}; c_{min} \leftarrow \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new}))$ 
13     $E \leftarrow E \cup \{(x_{min}, x_{new})\};$ 
14    foreach  $x_{near} \in X_{near}$  do // Rewire the tree
15      if  $\text{CollisionFree}(x_{new}, x_{near}) \wedge \text{Cost}(x_{new}) + c(\text{Line}(x_{new}, x_{near})) < \text{Cost}(x_{near})$ 
16        then  $x_{parent} \leftarrow \text{Parent}(x_{near});$ 
17         $E \leftarrow (E \setminus \{(x_{parent}, x_{near})\}) \cup \{(x_{new}, x_{near})\}$ 
17 return  $G = (V, E);$ 

```



RRT* Algorithm

1. Initialize



Algorithm 6: RRT*

```

1  $V \leftarrow \{x_{init}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{rand} \leftarrow \text{SampleFree}_i;$ 
4    $x_{nearest} \leftarrow \text{Nearest}(G = (V, E), x_{rand});$ 
5    $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand});$ 
6   if  $\text{ObstacleFree}(x_{nearest}, x_{new})$  then
7      $X_{near} \leftarrow \text{Near}(G = (V, E), x_{new}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}(V))/\text{card}(V))^{1/d}, \eta\});$ 
8      $V \leftarrow V \cup \{x_{new}\};$ 
9      $x_{min} \leftarrow x_{nearest}; c_{min} \leftarrow \text{Cost}(x_{nearest}) + c(\text{Line}(x_{nearest}, x_{new}));$ 
10    foreach  $x_{near} \in X_{near}$  do // Connect along a minimum-cost path
11      if  $\text{CollisionFree}(x_{near}, x_{new}) \wedge \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new})) < c_{min}$  then
12         $x_{min} \leftarrow x_{near}; c_{min} \leftarrow \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new}))$ 
13     $E \leftarrow E \cup \{(x_{min}, x_{new})\};$ 
14    foreach  $x_{near} \in X_{near}$  do // Rewire the tree
15      if  $\text{CollisionFree}(x_{new}, x_{near}) \wedge \text{Cost}(x_{new}) + c(\text{Line}(x_{new}, x_{near})) < \text{Cost}(x_{near})$ 
16        then  $x_{parent} \leftarrow \text{Parent}(x_{near});$ 
17         $E \leftarrow (E \setminus \{(x_{parent}, x_{near})\}) \cup \{(x_{new}, x_{near})\}$ 
17 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 10$

Step = 5

Start = (1,1)

Goal = (10.5, 10)



RRT* Algorithm

1. Initialize

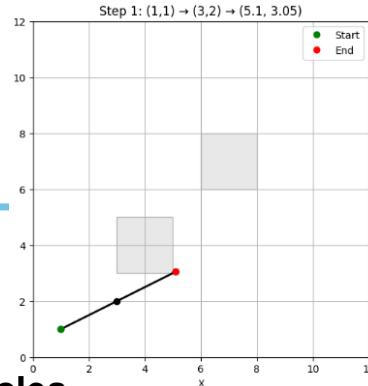
2. Generate Sample Particles

$N = 1$

Sample x_r : (3,2)

Dist.(sample, Start) = 2.24

Nearest(G , x_r) = Start



Algorithm 6: RRT*

```

1  $V \leftarrow \{x_{init}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{rand} \leftarrow \text{SampleFree}_i;$ 
4    $x_{nearest} \leftarrow \text{Nearest}(G = (V, E), x_{rand});$ 
5    $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand});$ 
6   if  $\text{ObstacleFree}(x_{nearest}, x_{new})$  then
7      $x_{near} \leftarrow \text{Near}(G = (V, E), x_{new}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}(V))/\text{card}(V))^{1/d}, \eta\});$ 
8      $V \leftarrow V \cup \{x_{new}\};$ 
9      $x_{min} \leftarrow x_{nearest}; c_{min} \leftarrow \text{Cost}(x_{nearest}) + c(\text{Line}(x_{nearest}, x_{new}));$ 
10    foreach  $x_{near} \in X_{near}$  do // Connect along a minimum-cost path
11      if  $\text{CollisionFree}(x_{near}, x_{new}) \wedge \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new})) < c_{min}$  then
12         $x_{min} \leftarrow x_{near}; c_{min} \leftarrow \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new}))$ 
13     $E \leftarrow E \cup \{(x_{min}, x_{new})\};$ 
14    foreach  $x_{near} \in X_{near}$  do // Rewire the tree
15      if  $\text{CollisionFree}(x_{new}, x_{near}) \wedge \text{Cost}(x_{new}) + c(\text{Line}(x_{new}, x_{near})) < \text{Cost}(x_{near})$ 
16        then  $x_{parent} \leftarrow \text{Parent}(x_{near});$ 
17         $E \leftarrow (E \setminus \{(x_{parent}, x_{near})\}) \cup \{(x_{new}, x_{near})\}$ 
17 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 10$

Step = 5

Start = (1,1)

Goal = (10.5, 10)



RRT* Algorithm

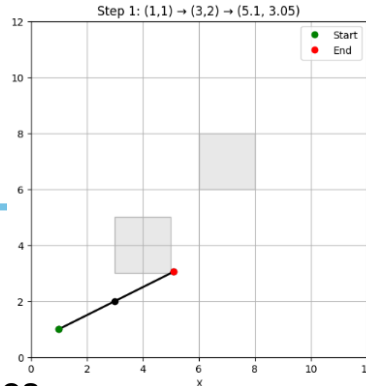
1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor

$N = 1$

Sample x_r : (3,2)

Dist.(sample, Start) = 2.24

Nearest(G , x_r) = x_n = Start



Algorithm 6: RRT*

```

1  $V \leftarrow \{x_{init}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{rand} \leftarrow \text{SampleFree}_i;$ 
4    $x_{nearest} \leftarrow \text{Nearest}(G = (V, E), x_{rand});$ 
5    $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand});$ 
6   if  $\text{ObstacleFree}(x_{nearest}, x_{new})$  then
7      $x_{near} \leftarrow \text{Near}(G = (V, E), x_{new}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}(V))/\text{card}(V))^{1/d}, \eta\});$ 
8      $V \leftarrow V \cup \{x_{new}\};$ 
9      $x_{min} \leftarrow x_{nearest}; c_{min} \leftarrow \text{Cost}(x_{nearest}) + c(\text{Line}(x_{nearest}, x_{new}));$ 
10    foreach  $x_{near} \in X_{near}$  do // Connect along a minimum-cost path
11      if  $\text{CollisionFree}(x_{near}, x_{new}) \wedge \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new})) < c_{min}$  then
12         $x_{min} \leftarrow x_{near}; c_{min} \leftarrow \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new}))$ 
13     $E \leftarrow E \cup \{(x_{min}, x_{new})\};$ 
14    foreach  $x_{near} \in X_{near}$  do // Rewire the tree
15      if  $\text{CollisionFree}(x_{new}, x_{near}) \wedge \text{Cost}(x_{new}) + c(\text{Line}(x_{new}, x_{near})) < \text{Cost}(x_{near})$ 
16        then  $x_{parent} \leftarrow \text{Parent}(x_{near});$ 
17         $E \leftarrow (E \setminus \{(x_{parent}, x_{near})\}) \cup \{(x_{new}, x_{near})\}$ 
17 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 10$

Step = 5

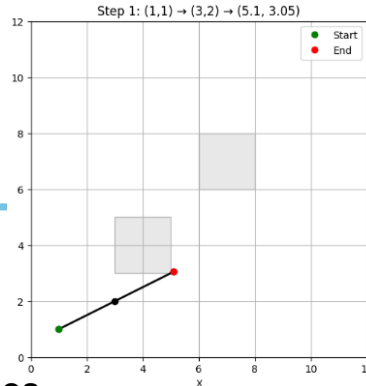
Start = (1,1)

Goal = (10.5, 10)



RRT* Algorithm

1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Steer Collision Free Path



Steer Function:

Find the unit vector from $X_n \rightarrow X_r$

Direction vector : (2,1)

Unit Direction : (2/2.24, 1/ 2.24)

Move in step size : (1,1) + 5 (~0.8, ~0.4)

$X_{new} : (5.1, 3.05)$

Algorithm 6: RRT*

```

1  $V \leftarrow \{x_{init}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{rand} \leftarrow \text{SampleFree}_i;$ 
4    $x_{nearest} \leftarrow \text{Nearest}(G = (V, E), x_{rand});$ 
5    $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand});$ 
6   if  $\text{ObstacleFree}(x_{nearest}, x_{new})$  then
7      $X_{near} \leftarrow \text{Near}(G = (V, E), x_{new}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}(V))/\text{card}(V))^{1/d}, \eta\});$ 
8      $V \leftarrow V \cup \{x_{new}\};$ 
9      $x_{min} \leftarrow x_{nearest}; c_{min} \leftarrow \text{Cost}(x_{nearest}) + c(\text{Line}(x_{nearest}, x_{new}));$ 
10    foreach  $x_{near} \in X_{near}$  do // Connect along a minimum-cost path
11      if  $\text{CollisionFree}(x_{near}, x_{new}) \wedge \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new})) < c_{min}$  then
12         $x_{min} \leftarrow x_{near}; c_{min} \leftarrow \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new}))$ 
13     $E \leftarrow E \cup \{(x_{min}, x_{new})\};$ 
14    foreach  $x_{near} \in X_{near}$  do // Rewire the tree
15      if  $\text{CollisionFree}(x_{new}, x_{near}) \wedge \text{Cost}(x_{new}) + c(\text{Line}(x_{new}, x_{near})) < \text{Cost}(x_{near})$ 
16        then  $x_{parent} \leftarrow \text{Parent}(x_{near});$ 
17         $E \leftarrow (E \setminus \{(x_{parent}, x_{near})\}) \cup \{(x_{new}, x_{near})\}$ 
17 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 10$

Step = 5

Start = (1,1)

Goal = (10.5, 10)

$N = 1$

Sample X_r : (3,2)

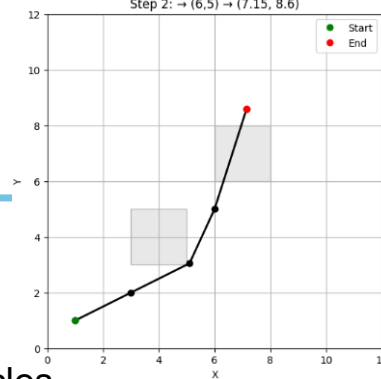
Dist.(sample, Start) = 2.24

Nearest(G , X_r) = Start



RRT* Algorithm

$(1,1) \rightarrow (5.1, 3.05) \rightarrow (9.6, 4.55)$



1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Steer Collision Free Path
5. Construct the Path
6. Post Processing

Construction & Rewiring Logic:

- Find the neighborhood around X_{new} : $(9.6, 4.55)$
 $r = \Gamma * (\log N / N)^{1/d}$
 $= 10 * (\log 2 / 2)^{1/2} = \sim 5.89$
- Set a better parent if it exists
- Optimize other neighbors if possible

Algorithm 6: RRT*

```

1  $V \leftarrow \{x_{init}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{rand} \leftarrow \text{SampleFree}_i;$ 
4    $x_{nearest} \leftarrow \text{Nearest}(G = (V, E), x_{rand});$ 
5    $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand});$ 
6   if  $\text{ObstacleFree}(x_{nearest}, x_{new})$  then
7      $X_{near} \leftarrow \text{Near}(G = (V, E), x_{new}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}(V))/\text{card}(V))^{1/d}, \eta\});$ 
8      $V \leftarrow V \cup \{x_{new}\};$ 
9      $x_{min} \leftarrow x_{nearest}; c_{min} \leftarrow \text{Cost}(x_{nearest}) + c(\text{Line}(x_{nearest}, x_{new}));$ 
10    foreach  $x_{near} \in X_{near}$  do // Connect along a minimum-cost path
11      if  $\text{CollisionFree}(x_{near}, x_{new}) \wedge \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new})) < c_{min}$  then
12         $x_{min} \leftarrow x_{near}; c_{min} \leftarrow \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new}))$ 
13     $E \leftarrow E \cup \{(x_{min}, x_{new})\};$ 
14    foreach  $x_{near} \in X_{near}$  do // Rewire the tree
15      if  $\text{CollisionFree}(x_{new}, x_{near}) \wedge \text{Cost}(x_{new}) + c(\text{Line}(x_{new}, x_{near})) < \text{Cost}(x_{near})$ 
16        then  $x_{parent} \leftarrow \text{Parent}(x_{near});$ 
17         $E \leftarrow (E \setminus \{(x_{parent}, x_{near})\}) \cup \{(x_{new}, x_{near})\}$ 
17 return  $G = (V, E);$ 

```

$d = 2$

$\Gamma = 10$

Step = 5

Start = (1,1)

Goal = (10.5, 10)

$N = 2$

Sample X_r : (8,4)

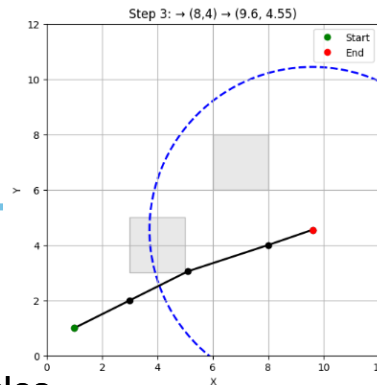
Dist.(sample, G) = 3.05

Nearest(G , X_r) = (5.1, 3.05)



RRT* Algorithm

$(1,1) \rightarrow (5.1, 3.05) \rightarrow (9.6, 4.55)$



1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Steer Collision Free Path
5. Construct the Path
6. Post Processing

Construction & Rewiring Logic:
For every neighbor check :

7. Goal Test
8. Repeat

$\text{Cost}((5.1, 3.05)) < \text{Cost}((1,1) \rightarrow (9.6, 4.55))$
 $+ \text{Dist}((5.1, 3.05), (9.6, 4.55))$
 $\sim 3.84 < \sim 9.3 + 4.74$
 No Rewiring is triggered

Algorithm 6: RRT*

```

1  $V \leftarrow \{x_{\text{init}}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{\text{rand}} \leftarrow \text{SampleFree}_i;$ 
4    $x_{\text{nearest}} \leftarrow \text{Nearest}(G = (V, E), x_{\text{rand}});$ 
5    $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{nearest}}, x_{\text{rand}});$ 
6   if  $\text{ObstacleFree}(x_{\text{nearest}}, x_{\text{new}})$  then
7      $x_{\text{near}} \leftarrow \text{Near}(G = (V, E), x_{\text{new}}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}(V))/\text{card}(V))^{1/d}, \eta\});$ 
8      $V \leftarrow V \cup \{x_{\text{new}}\};$ 
9      $x_{\text{min}} \leftarrow x_{\text{nearest}}; c_{\text{min}} \leftarrow \text{Cost}(x_{\text{nearest}}) + c(\text{Line}(x_{\text{nearest}}, x_{\text{new}}));$ 
10    foreach  $x_{\text{near}} \in X_{\text{near}}$  do // Connect along a minimum-cost path
11      if  $\text{CollisionFree}(x_{\text{near}}, x_{\text{new}}) \wedge \text{Cost}(x_{\text{near}}) + c(\text{Line}(x_{\text{near}}, x_{\text{new}})) < c_{\text{min}}$  then
12         $x_{\text{min}} \leftarrow x_{\text{near}}; c_{\text{min}} \leftarrow \text{Cost}(x_{\text{near}}) + c(\text{Line}(x_{\text{near}}, x_{\text{new}}))$ 
13     $E \leftarrow E \cup \{(x_{\text{min}}, x_{\text{new}})\};$ 
14    foreach  $x_{\text{near}} \in X_{\text{near}}$  do // Rewire the tree
15      if  $\text{CollisionFree}(x_{\text{new}}, x_{\text{near}}) \wedge \text{Cost}(x_{\text{new}}) + c(\text{Line}(x_{\text{new}}, x_{\text{near}})) < \text{Cost}(x_{\text{near}})$ 
16        then  $x_{\text{parent}} \leftarrow \text{Parent}(x_{\text{near}});$ 
17         $E \leftarrow (E \setminus \{(x_{\text{parent}}, x_{\text{near}})\}) \cup \{(x_{\text{new}}, x_{\text{near}})\}$ 
17 return  $G = (V, E);$ 
  
```

$N = 2$

Sample x_r : (8,4)

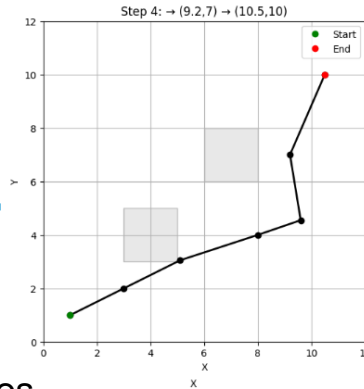
Dist.(sample, G) = 3.05

Nearest(G , x_r) = (5.1, 3.05)



RRT* Algorithm

$(1,1) \rightarrow (5.1, 3.05) \rightarrow (9.6, 4.55) \dots$



1. Initialize
2. Generate Sample Particles
3. Search Nearest Neighbor
4. Steer Collision Free Path
5. Construct the Path
6. Post Processing
7. Goal Test
8. Repeat

Algorithm 6: RRT*

```

1  $V \leftarrow \{x_{init}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{rand} \leftarrow \text{SampleFree}_i;$ 
4    $x_{nearest} \leftarrow \text{Nearest}(G = (V, E), x_{rand});$ 
5    $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand});$ 
6   if  $\text{ObstacleFree}(x_{nearest}, x_{new})$  then
7      $X_{near} \leftarrow \text{Near}(G = (V, E), x_{new}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}(V))/\text{card}(V))^{1/d}, \eta\});$ 
8      $V \leftarrow V \cup \{x_{new}\};$ 
9      $x_{min} \leftarrow x_{nearest}; c_{min} \leftarrow \text{Cost}(x_{nearest}) + c(\text{Line}(x_{nearest}, x_{new}));$ 
10    foreach  $x_{near} \in X_{near}$  do // Connect along a minimum-cost path
11      if  $\text{CollisionFree}(x_{near}, x_{new}) \wedge \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new})) < c_{min}$  then
12         $x_{min} \leftarrow x_{near}; c_{min} \leftarrow \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new}))$ 
13     $E \leftarrow E \cup \{(x_{min}, x_{new})\};$ 
14    foreach  $x_{near} \in X_{near}$  do // Rewire the tree
15      if  $\text{CollisionFree}(x_{new}, x_{near}) \wedge \text{Cost}(x_{new}) + c(\text{Line}(x_{new}, x_{near})) < \text{Cost}(x_{near})$ 
16        then  $x_{parent} \leftarrow \text{Parent}(x_{near});$ 
17         $E \leftarrow (E \setminus \{(x_{parent}, x_{near})\}) \cup \{(x_{new}, x_{near})\}$ 
17 return  $G = (V, E);$ 

```

$N = 2$

Sample x_r : (8,4)

Dist.(sample, G) = 3.05

Nearest(G , x_r) = (5.1, 3.05)

Probabilistic Road Maps (PRM)



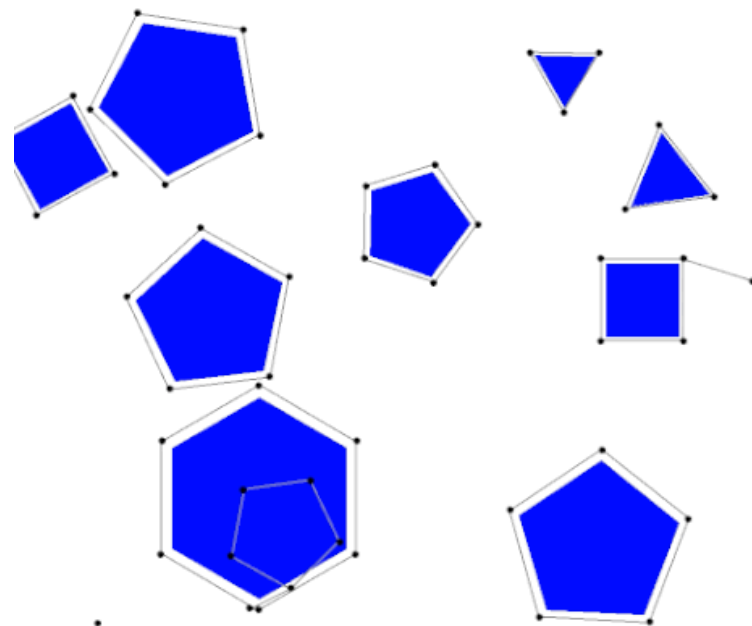
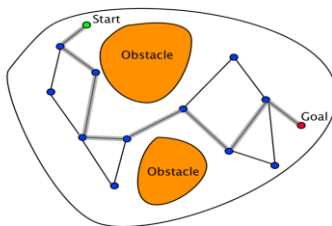
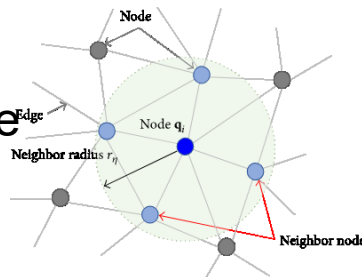
Learning (Preprocess) Phase

Random Sampling

Roadmap Construction

Forest

Querying Phase



To be continued in the next class

Required Reading

- T2 Chapter 12

<https://erc-bpgc.github.io/handbook/>

Research Papers and Web Resources

Next Class Plan

- ✓ Sampling Based Planner – Part 2
- ✓ Evolutionary Algorithms